Project Report: Maritime Vessel Tracking, Port Analytics, and Safety Visualization Platform

---

1. Introduction

1.1 Background & Motivation

The global maritime industry relies heavily on accurate, real-time data to ensure the efficient movement of goods. However, stakeholders such as shipping companies, port authorities, and insurers often face challenges related to fragmented data sources, lack of real-time visibility, and the inability to visualize safety risks like piracy or severe weather in the context of active voyages. Existing solutions are often siloed or prohibitively expensive .The global maritime industry relies heavily on accurate, real-time data to ensure the efficient and safe movement of goods across oceans. However, key stakeholders such as shipping companies, port authorities, insurers, and logistics providers often face significant challenges. These difficulties stem from fragmented data sources, which are typically spread across disparate legacy systems, leading to a critical lack of real-time visibility into the status and location of vessels. Furthermore, a major gap exists in the ability to effectively visualize and assess critical safety risks, such as active piracy threats, severe weather events, or geopolitical hotspots, in the direct context of current and planned voyage routes. Existing technological solutions designed to address these issues are frequently siloed, focusing on only one aspect of maritime operations (e.g., vessel tracking or weather forecasting), or are prohibitively expensive and complex for broader adoption across the industry, particularly for smaller operators.

1.2 Project Objective

The primary objective of this project is to develop a full-stack web platform that democratizes access to maritime intelligence. By aggregating open-source maritime, weather, and trade data, the platform provides a unified dashboard for interactive vessel tracking, cargo classification, and risk assessment.

1.3 Scope

The platform is designed to serve three key user groups:

- **Shipping Companies:** For fleet tracking, allowing them to monitor the real-time location and status of all their vessels globally, and voyage optimization, helping them select the most fuel-efficient and timely routes while avoiding known hazards or restricted areas.
- **Port Authorities:** For monitoring congestion, providing a comprehensive overview of vessel density and berthing availability, and managing arrivals/departures, enabling smooth scheduling and efficient resource allocation for pilotage, tugs, and cargo operations.
- ****Maritime Insurers:** For assessing risks related to piracy zones, offering up-to-date threat assessments and historical incident data to accurately price premiums for high-risk transits, and weather conditions, providing predictive modeling for storms and severe sea states that could lead to claims for damage or delays.

2. System Analysis and Design

2.1 System Architecture

The application follows a modern, decoupled **Client-Server Architecture** :

- **Frontend (Client Layer):** Built using **React.js**, a popular JavaScript library for building user interfaces, the frontend is responsible for rendering the interactive user interface. It utilizes specialized libraries: **Leaflet.js** for handling dynamic and interactive mapping functionalities to display vessel locations and routes, and **Recharts** for clear, customizable data visualization of statistics like port traffic and voyage trends. It communicates securely and efficiently with the backend via RESTful API calls using the promise-based HTTP client **Axios**.

- **Backend (Server Layer):** Developed with **Python Django**, a high-level Python web framework that encourages rapid development and clean, pragmatic design, and **Django REST Framework (DRF)** for building robust and scalable web APIs. This layer handles all core business logic, including complex data processing, authentication, authorization, and managing the API endpoints. Crucially, it acts as a secure proxy to fetch real-time data from external third-party APIs (such as MarineTraffic, AIS Hub, and NOAA) to centrally protect sensitive API keys and manage rate limits, preventing client-side exposure.

- **Database Layer:** A robust, relational **PostgreSQL** database serves as the persistent, centralized storage system. It was migrated from a file-based SQLite database used during initial development to ensure better concurrency, data integrity, and scalability for a production environment. It stores critical structured data, including user profiles, detailed vessel metadata, comprehensive port statistics, and historical voyage logs for analysis.

2.2 Technology Stack

- **Programming Languages:** Python (Backend), JavaScript (Frontend) .
- **Web Frameworks:** Django, Django REST Framework (DRF), React.js .
- **Data Visualization:** Leaflet.js (Maps), Recharts (Analytics) .
- **Authentication:** JWT (JSON Web Tokens) via `djangorestframework-simplejwt`.
- **External APIs:** MarineTraffic/AIS Hub (Vessel Data), NOAA (Weather), UNCTAD (Trade Stats) .**Programming Languages:** Python (Backend), JavaScript (Frontend) .

- **Web Frameworks:** Django, Django REST Framework (DRF), React.js .
- **Databases:** PostgreSQL, SQLite.
- **Version Control:** Git, GitHub.
- **Data Visualization:** Leaflet.js (Maps), Recharts (Analytics) .
- **Authentication:** JWT (JSON Web Tokens) via djangorestframework-simplejwt.

External APIs: MarineTraffic/AIS Hub (Vessel Data), NOAA (Weather), UNCTAD (Trade Stats) .

3. Database Design & Schema

The database is structured to support relational data integrity across users, assets, and events. Based on the system schema, the key entities include :

3.1 Users Table (`users`)

Manages authentication and role-based access.

- `id`: Primary Key.
- `username`, `email`: Unique identifiers for login.
- `password`: Hashed security string.
- `role`: Defines user permissions (Admin, Operator, Analyst) .

3.2 Vessels Table (`vessels`)

Stores static and dynamic information about ships.

- `imo_number`: Unique International Maritime Organization ID.
- `name`: Vessel name.
- `type`: Classification (e.g., Cargo, Tanker).
- `flag`: Country of registration.
- `last_position_lat`, `last_position_lon`: Most recent coordinates.
- `operator`: The company managing the vessel .

3.3 Ports Table (`ports`)

Stores global port data for analytics.

- `name`, `location`, `country`: Geographical details.
- `congestion_score`: A calculated metric indicating traffic density.
- `avg_wait_time`: Performance metric for insurers and logistics planners .

3.4 Events Table (`events`)

Logs significant occurrences for historical replay and alerts.

- `event_type`: Category (e.g., "Storm", "Piracy", "Delay").
- `location`: Coordinates or region name.
- `timestamp`: Time of occurrence .

4. Implementation Details (Milestones)

4.1 Milestone 1: Foundation & Security

The initial phase focused on setting up a secure environment.

- **RBAC Implementation:** We created a custom permission class in Django using the `rest_framework.permissions.BasePermission` to restrict access based on user roles (e.g., Operator, Analyst). This fine-grained control ensures the principle of least privilege. For example, Operator users can edit and update vessel details, while Analysts have read-only access to operational dashboards and aggregated data reports.
- **JWT Authentication:** Implemented a stateless authentication mechanism using Django Rest Framework Simple JWT. Upon successful login, the server issues a short-lived access token and a longer-lived refresh token. This system allows for secure session management without the overhead or security risks associated with server-side session storage, as the tokens are validated directly by the application layer. The refresh token is used to securely obtain a new access token when the current one expires.Implemented a stateless authentication mechanism where the server issues an access and refresh token upon login, ensuring secure session management without server-side session storage.

4.2 Milestone 2: Live Tracking Core

This milestone delivered the primary feature of the platform.

- **Map Integration:** We integrated **Leaflet.js** to create a responsive map. Custom markers (ship icons) were implemented to differentiate vessels visually.
- **API Aggregation:** A background service (or proxy view) was created in Django to fetch data from the MarineTraffic API, normalize it, and serve it to the frontend. This prevents Cross-Origin Resource Sharing (CORS) issues and hides API credentials.
- **Filtering Logic:** A complex filtering state was built in React, allowing users to slice data by **Vessel Type** (Container, LNG, Fishing) or **Flag State**. This logic executes client-side for immediate feedback .

This milestone delivered the primary feature of the platform.

- **Map Integration:** We integrated **Leaflet.js** to create a responsive map. Custom markers (ship icons) were implemented to differentiate vessels visually. **Real-time vessel position updates** were managed using WebSockets for a live tracking experience.
- **API Aggregation:** A background service (or proxy view) was created in Django to

fetch data from the MarineTraffic API, normalize it, and serve it to the frontend. This prevents Cross-Origin Resource Sharing (CORS) issues and hides API credentials. **Data caching with Redis** was implemented to reduce API call frequency and improve load times.

- **Filtering Logic:** A complex filtering state was built in React, allowing users to slice data by **Vessel Type** (Container, LNG, Fishing) or **Flag State**. This logic executes client-side for immediate feedback. **Performance optimization** for filtering was achieved by debouncing input and utilizing React's `useMemo` hook.

4.3 Milestone 3: Analytics & Safety Intelligence

The focus shifted to data value extraction and safety.

- **Port Congestion:** We simulated UNCTAD trade data integration by creating visual charts (Area and Bar charts) that display Import/Export volumes and average wait times per port. *This feature was developed using D3.js for rendering the charts, allowing for interactive filtering by time range and port name.*
- **Safety Overlays:** Using Leaflet's Circle and Polygon components, we drew "High Risk" zones on the map. Red zones indicate piracy areas (e.g., Gulf of Aden), while orange zones represent active storm systems derived from NOAA data. *A REST API was set up to fetch and dynamically update the storm system data every 6 hours.*
- **Notification System:** A notification center was built to alert users of critical events (e.g., "Vessel entering Piracy Zone"). These alerts are persisted in the user's local session for continuity. *The system utilizes a pub/sub pattern to decouple the alert generation from the display, ensuring real-time event processing.*

5. Results and Discussion

5.1 User Interface

The final dashboard presents a unified view:

- **Top Navigation:** Provides quick access to Modules (Vessels, Ports, Tracking).
- **Main Workspace:** A split-screen layout with the Live Map on one side and detailed Data Panels on the other.
- **Alerts:** A floating notification panel ensures critical updates are never missed.The final dashboard presents a unified view:
- **Top Navigation:** Provides quick access to Modules (Vessels, Ports, Tracking) and includes a user profile and settings menu.
- **Main Workspace:** A split-screen layout with the Live Map on one side, offering real-time vessel positions and weather overlays, and detailed Data Panels on the other, which are context-sensitive based on the selected module.

Alerts: A floating notification panel ensures critical updates are never missed, categorized by severity (Informational, Warning, Critical) and easily filtered or dismissed.

5.2 Performance

By offloading map rendering to the client (React) and handling data fetching asynchronously, the application maintains a smooth 60fps frame rate even when tracking multiple vessels. This client-side rendering approach, utilizing libraries optimized for performance, ensures a highly responsive user experience with minimal UI lag. The backend serves as a lightweight API gateway, primarily responsible for secure data aggregation and low-latency transmission of real-time vessel updates via WebSockets.

6. Conclusion & Future Scope

This project successfully demonstrates the capability to integrate disparate maritime data sources into a cohesive intelligence platform. We have met the core objectives of live tracking, metadata visualization, and basic risk assessment.

Future Work (Milestone 4):

- **Historical Replay:** Implementing a "Time Slider" to replay a vessel's voyage history for audit and compliance purposes .
- **Predictive Analytics:** Using machine learning on the historical `events` data to predict port congestion before it happens.
- **Cloud Deployment:** Deploying the Dockerized application to AWS or Render for global accessibility .